

```
#!/usr/bin/env python3
```

```
"""
```

```
watch_unclaims.py
```

Watches <https://map.berrysmp.net>'s public squaremap data for GriefPrevention claims that disappear between runs (i.e. get auto-unclaimed after inactivity, or manually deleted/abandoned by their owner).

Includes a GRACE PERIOD so that someone removing their claim and re-adding it minutes later (e.g. while rebuilding/adjusting their base) doesn't trigger a false alert.

HOW IT WORKS

- Squaremap doesn't expose per-claim IDs, so each claim is fingerprinted by (owner, x1, z1, x2, z2) - stable as long as the claim exists.
- Every run, the script fetches the current claim list and compares it to the last saved snapshot.
- If a claim disappears, it's NOT reported immediately. It goes into a "pending removal" list with a timestamp.
- On later runs, if that same owner has re-claimed the same spot (exact match, OR a new claim that overlaps the old one's area), the pending removal is cleared silently - no alert.
- Only if the claim is STILL missing after GRACE_PERIOD_MINUTES does the script actually alert you that it's unclaimed.

USAGE

```
python3 watch_unclaims.py          # run once, print results
python3 watch_unclaims.py --loop 900 # run forever, check every 900s (15 min)
```

The first run just saves a baseline snapshot (nothing to compare against yet).

RUNNING IT CONTINUOUSLY

- 1) `python3 watch\_unclaims.py --loop 900` left running in a terminal / tmux / s
- 2) A cron job that runs `python3 watch\_unclaims.py` every 15 minutes (no --loop
- 3) A Windows Task Scheduler task doing the same on Windows.

NOTIFICATIONS

By default this just prints to the console and appends to unclaims_log.txt.
If you want a push notification (e.g. to Discord), set DISCORD_WEBHOOK_URL below - alerts will be posted there too.

```
"""
```

```
import json
import time
import urllib.request
import argparse
from pathlib import Path
from datetime import datetime, timezone

MARKERS_URL = "https://map.berrysmp.net/tiles/minecraft_overworld/markers.json"
STATE_FILE = Path(__file__).parent / "watcher_state.json"
LOG_FILE = Path(__file__).parent / "unclaims_log.txt"

# How long a claim has to stay gone before we actually alert you.
GRACE_PERIOD_MINUTES = 20

# Optional: paste a Discord webhook URL here to also get alerts posted to a channel
# Leave as None to skip. e.g. "https://discord.com/api/webhooks/xxxx/yyyy"
DISCORD_WEBHOOK_URL = None

# Optional (recommended): a ntfy.sh topic name for real push notifications to your
# phone. Pick any hard-to-guess topic name (it's public - anyone who knows the exact
# name can subscribe), install the ntfy app, subscribe to this same topic, and alerts
# will pop up on your phone like a normal app notification. Leave as None to skip
# e.g. "berrysmp-unclaims-x7q2f9"
NTFY_TOPIC = "berrysmp-unclaims-826hdhe"

def fetch_claims():
    """Fetch current claims as {fingerprint: claim_info}."""
    req = urllib.request.Request(MARKERS_URL, headers={"User-Agent": "Mozilla/5.0"})
    with urllib.request.urlopen(req, timeout=30) as resp:
        data = json.loads(resp.read().decode())

    gp = next((x for x in data if x["id"] == "griefprevention"), None)
    if gp is None:
        raise RuntimeError("Couldn't find 'griefprevention' marker set - map structure")

    claims = {}
    for marker in gp.get("markers", []):
        popup = marker.get("popup", "")
        owner = extract_owner(popup)
        pts = marker.get("points", [])
        if len(pts) != 2 or owner is None:
            continue
        x1, z1 = pts[0]["x"], pts[0]["z"]
        x2, z2 = pts[1]["x"], pts[1]["z"]
```

```

        xa, xb = sorted((x1, x2))
        za, zb = sorted((z1, z2))
        fingerprint = f"{owner}|{xa}|{za}|{xb}|{zb}"
        claims[fingerprint] = {
            "owner": owner,
            "x1": xa, "z1": za, "x2": xb, "z2": zb,
            "width": xb - xa,
            "height": zb - za,
        }
    return claims

def extract_owner(popup_html):
    marker = 'Claim Owner: <span style="font-weight:bold;">'
    start = popup_html.find(marker)
    if start == -1:
        return None
    start += len(marker)
    end = popup_html.find("</span>", start)
    if end == -1:
        return None
    owner = popup_html[start:end].strip()
    return owner if owner else None

def boxes_overlap(a, b):
    """True if two claim bounding boxes overlap at all."""
    return not (a["x2"] < b["x1"] or b["x2"] < a["x1"] or a["z2"] < b["z1"] or b[

def load_state():
    if STATE_FILE.exists():
        return json.loads(STATE_FILE.read_text())
    return {"claims": None, "pending_removals": {}}

def save_state(claims, pending_removals):
    STATE_FILE.write_text(json.dumps({"claims": claims, "pending_removals": pendi

def log_and_print(message):
    print(message)
    with open(LOG_FILE, "a") as f:
        f.write(message + "\n")

def notify_ntfy(message):

```

```

if not NTFY_TOPIC:
    return
try:
    req = urllib.request.Request(
        f"https://ntfy.sh/{NTFY_TOPIC}",
        data=message.encode("utf-8"),
        headers={
            "Title": "Claim unclaimed!",
            "Priority": "high",
            "Tags": "warning",
        },
        method="POST",
    )
    urllib.request.urlopen(req, timeout=15)
except Exception as e:
    print(f"    (ntfy notify failed: {e})")

def notify_discord(message):
    if not DISCORD_WEBHOOK_URL:
        return
    try:
        payload = json.dumps({"content": message}).encode()
        req = urllib.request.Request(
            DISCORD_WEBHOOK_URL,
            data=payload,
            headers={"Content-Type": "application/json"},
            method="POST",
        )
        urllib.request.urlopen(req, timeout=15)
    except Exception as e:
        print(f"    (Discord notify failed: {e})")

def run_once():
    now = datetime.now(timezone.utc)
    timestamp = now.strftime("%Y-%m-%d %H:%M:%S")
    current = fetch_claims()
    state = load_state()
    previous = state["claims"]
    pending = state["pending_removals"] # fingerprint -> {claim, first_missing_a

    if previous is None:
        log_and_print(f"[{timestamp}] First run - saved baseline of {len(current)}")
        save_state(current, {})
        return

```

```

newly_disappeared = {k: v for k, v in previous.items() if k not in current and
appeared = {k: v for k, v in current.items() if k not in previous}

# Add newly-disappeared claims to the pending list (don't alert yet)
for fp, c in newly_disappeared.items():
    pending[fp] = {"claim": c, "first_missing_at": now.isoformat()}

# Check pending removals: did they come back (exact or overlapping same owner)
resolved = []
confirmed_gone = []
for fp, entry in pending.items():
    c = entry["claim"]

    # Case 1: exact same claim reappeared
    if fp in current:
        resolved.append(fp)
        continue

    # Case 2: same owner re-claimed an overlapping area (different exact corner)
    reclaimed = any(
        new_c["owner"] == c["owner"] and boxes_overlap(c, new_c)
        for new_c in current.values()
    )
    if reclaimed:
        resolved.append(fp)
        continue

    # Still gone - check if grace period has elapsed
    first_missing = datetime.fromisoformat(entry["first_missing_at"])
    elapsed_minutes = (now - first_missing).total_seconds() / 60
    if elapsed_minutes >= GRACE_PERIOD_MINUTES:
        confirmed_gone.append(fp)

for fp in resolved:
    c = pending[fp]["claim"]
    log_and_print(
        f"[{timestamp}] (no alert) {c['owner']}'s claim at ({c['x1']},{c['z1']} "
        f"was re-claimed before the grace period ended - ignoring."
    )
    del pending[fp]

for fp in confirmed_gone:
    c = pending[fp]["claim"]
    msg = (
        f"[{timestamp}] UNCLAIMED: {c['owner']}'s claim "
        f"({c['width']}{c['height']}) at ({c['x1']},{c['z1']}) to ({c['x2']} "
        f"has been gone for {GRACE_PERIOD_MINUTES}+ min - go check it out!"
    )

```

```

    )
    log_and_print(msg)
    notify_discord(msg)
    notify_ntfy(msg)
    del pending[fp]

if not newly_disappeared and not confirmed_gone and not resolved:
    log_and_print(f"[{timestamp}] Checked {len(current)} claims - no changes.
if pending:
    log_and_print(f"[{timestamp}]    ({len(pending)} claim(s) currently in gra
if appeared:
    log_and_print(f"[{timestamp}]    ({len(appeared)} new claim(s) also appear

save_state(current, pending)

def main():
    parser = argparse.ArgumentParser(description="Watch berrysmp claims for ones
    parser.add_argument("--loop", type=int, default=0,
                        help="Run forever, checking every N seconds (e.g. 900 fo
    args = parser.parse_args()

    if args.loop > 0:
        log_and_print(f"Starting watcher, checking every {args.loop} seconds (gra
        while True:
            try:
                run_once()
            except Exception as e:
                log_and_print(f"[ERROR] {e}")
                time.sleep(args.loop)
    else:
        run_once()

if __name__ == "__main__":
    main()

```